Point-to-Point and Multi-Point Navigation and Obstacle Avoidance

Before initiating navigation, it's essential to enable the robot to map. You can find detailed guidance on this process in “12. Mapping Lesson/ Lesson 4 slam_toolbox Mapping Algorithm”.

1. Robot Operations

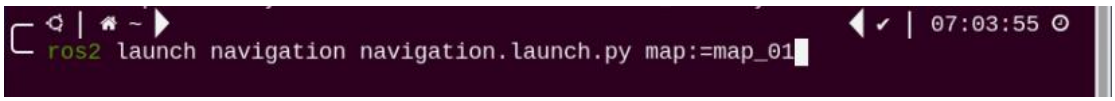
1）Click-on  to open the command line terminal.

2）Execute the command to disable the app auto-start service:

```
~/.stop_ros.sh
```

3）Enter the following command to start the navigation service and press Enter:

```
ros2 launch navigation navigation.launch.py map:=map_01
```



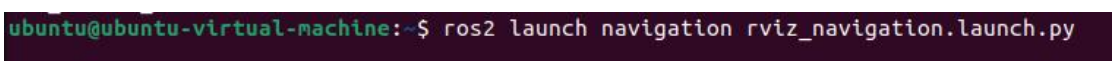
The "map_01" at the end of the command is the map name. You can modify this parameter according to your needs. The map is stored at "/home/ubuntu/ros2_ws/src/slam/maps".

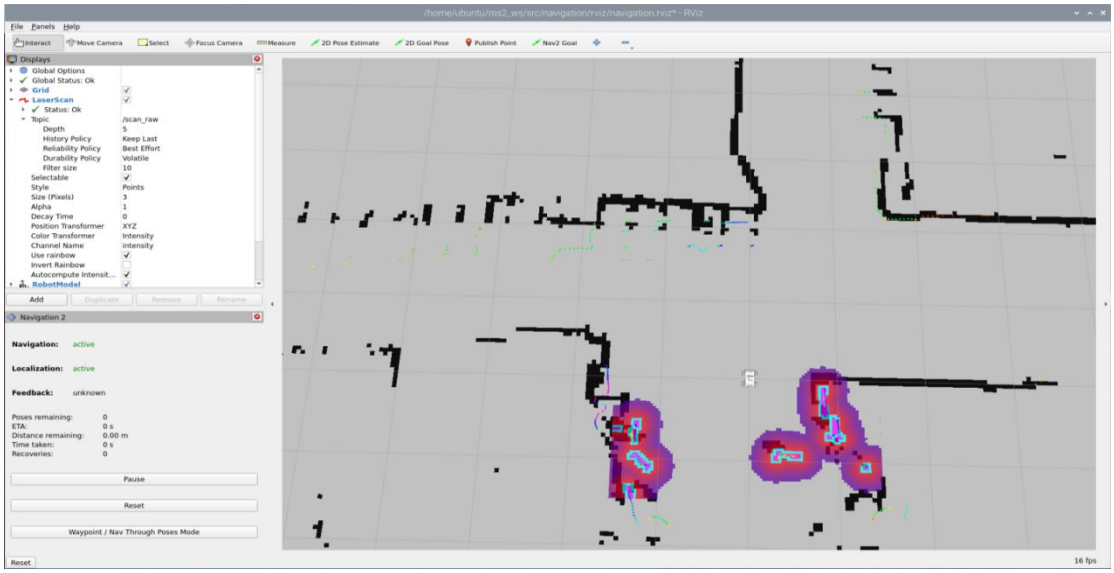
2. Virtual Machine Operations

1）Click-on  to open the command-line terminal on the virtual machine.

2）Enter the following command to start the RViz tool for navigation display:

```
ros2 launch navigation rviz_navigation.launch.py
```

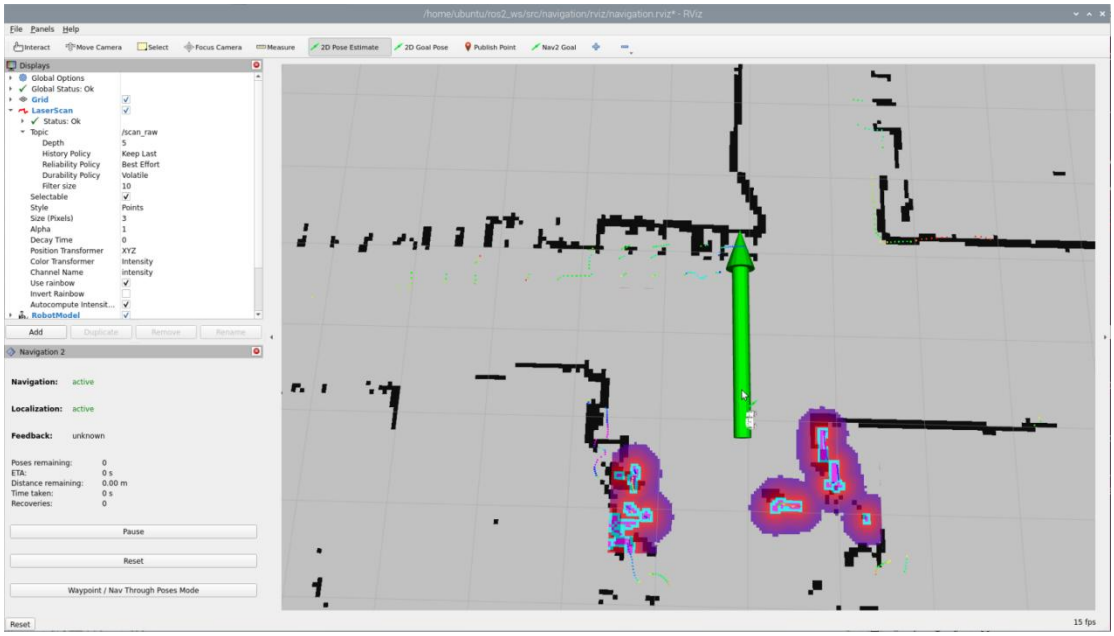




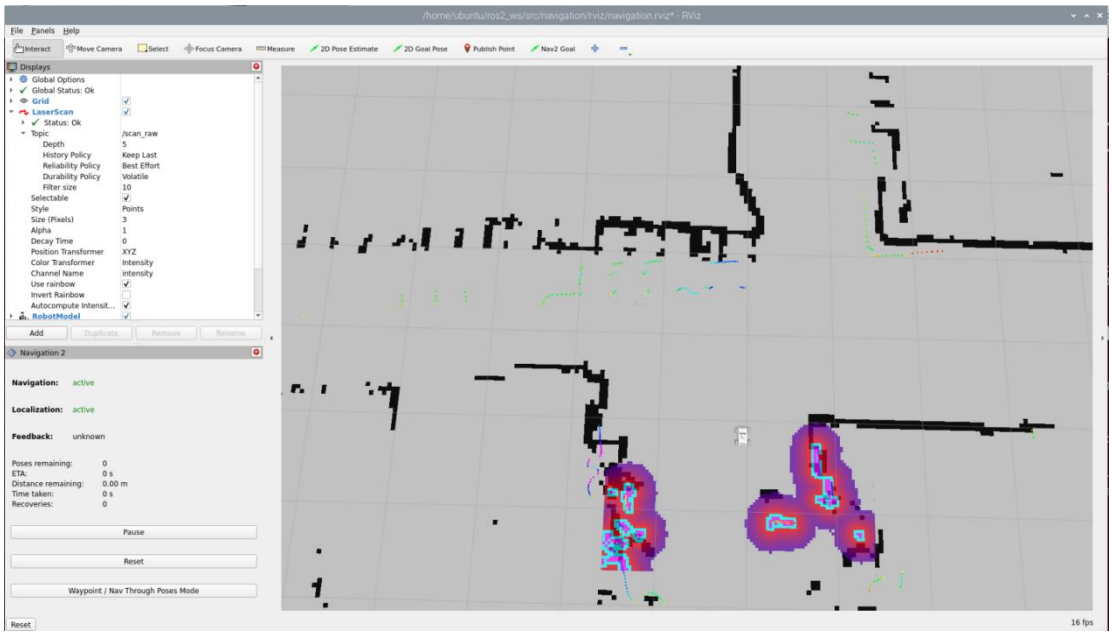
2.1 Point-to-Point Navigation

In the software menu bar, "2D Pose Estimate" is used to set the initial position of the JetRover robot; "2D Nav Goal" is used to set a single target point for the robot; "Publish Point" is used to set multiple target points for the robot.

1) Click  to set the initial position of the robot. On the map interface, choose a position, click, and drag the mouse to select the robot's pose.

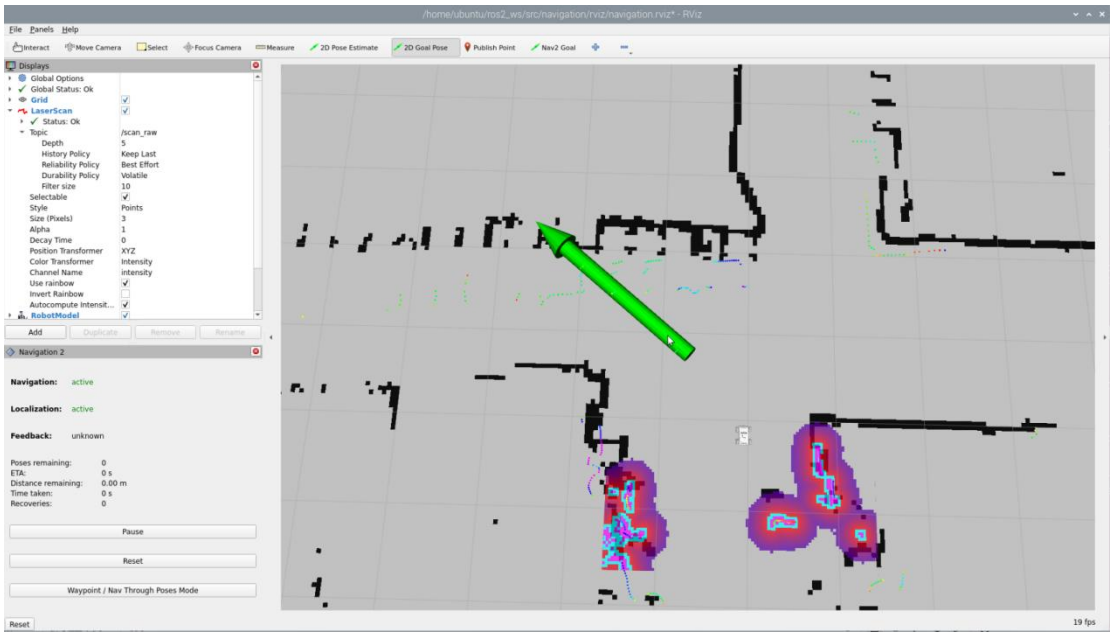


2) After setting the initial position of the robot, the effect is as shown in the figure below:

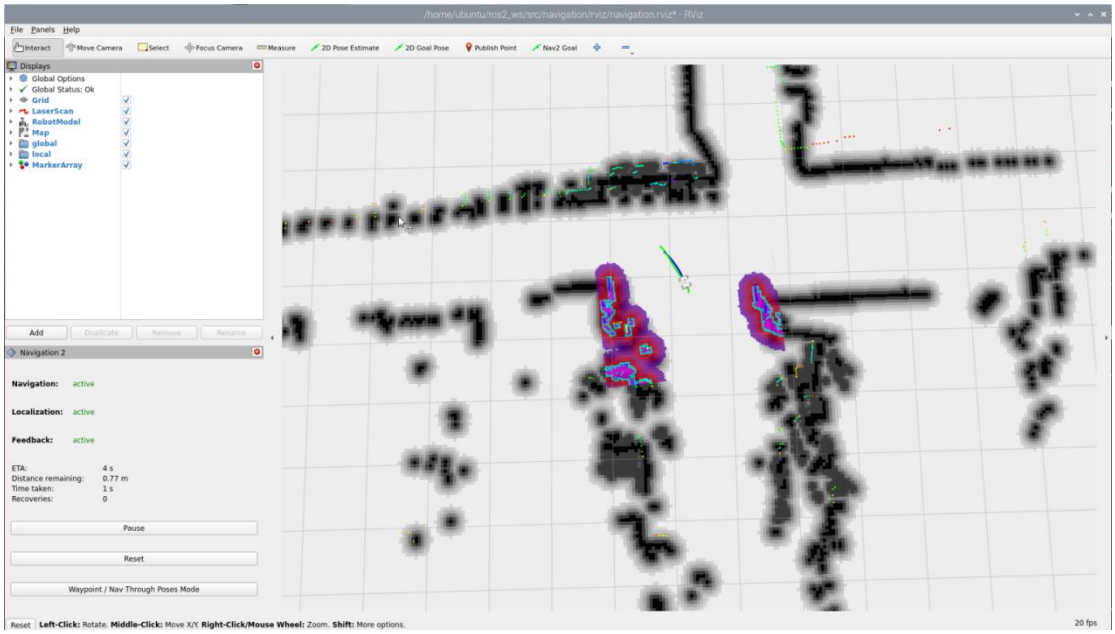


3) Click 

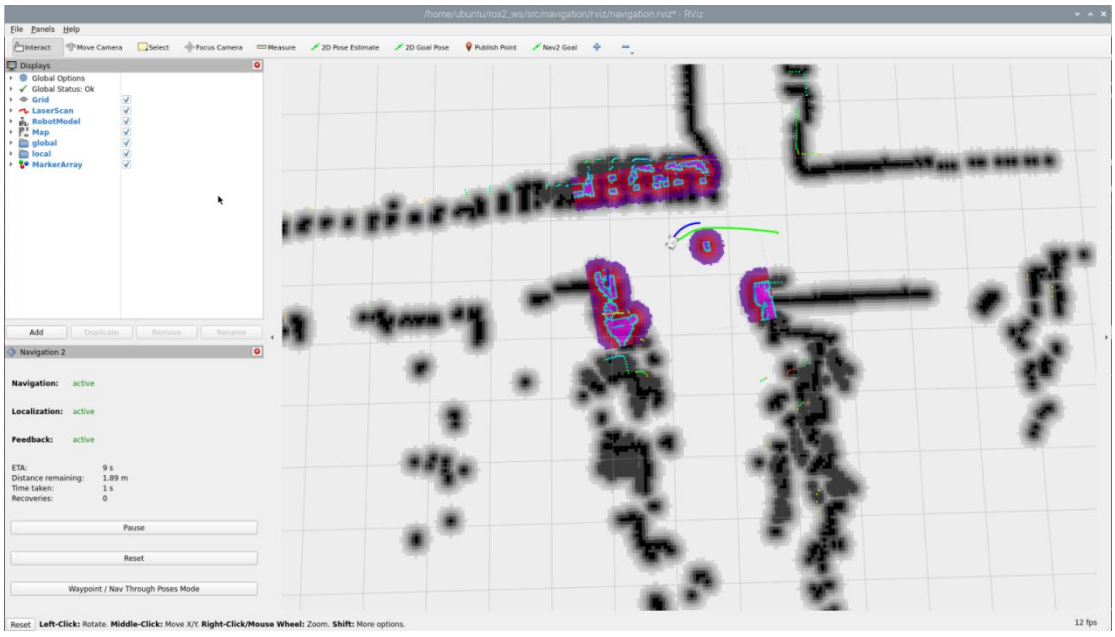
and select a location on the map interface as the target point. Click the mouse once at that point. After selection, the robot will automatically generate a route and move to the target point.



4) After confirming the target point, the map will display two paths: the green line represents the direct path between the robot and the target point, and the dark blue line represents the planned path of the robot.

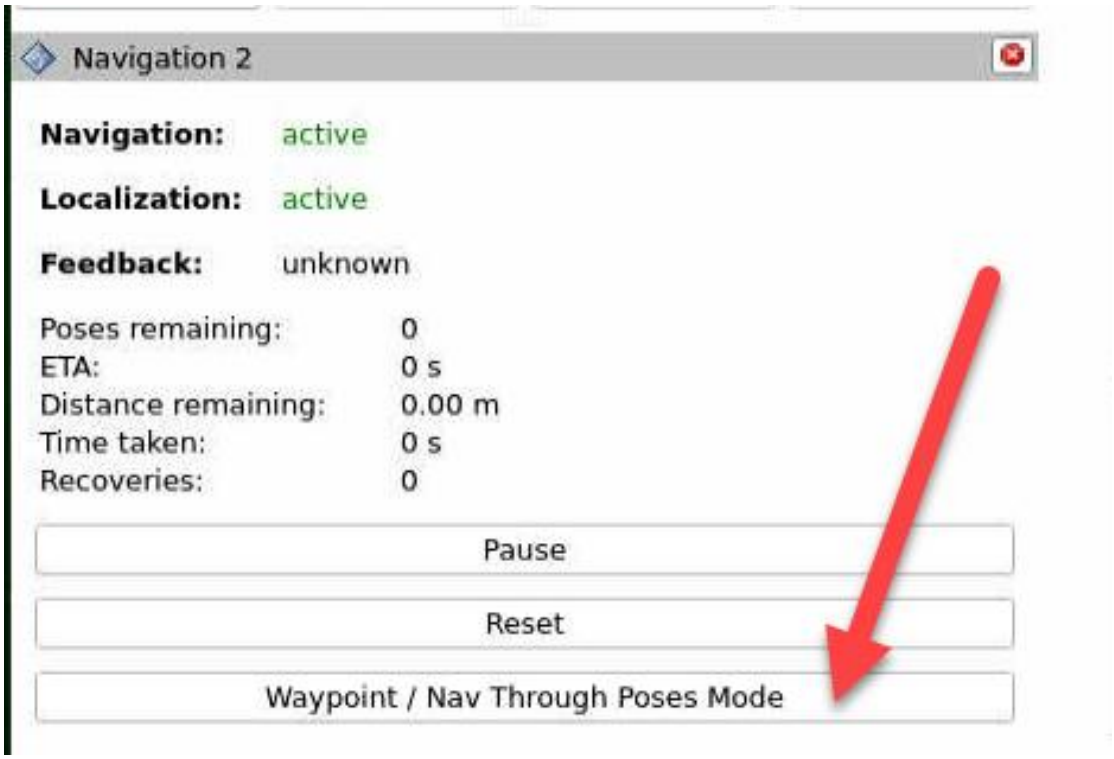


5) When encountering obstacles, the robot will bypass them and continuously adjust its posture and travel route.



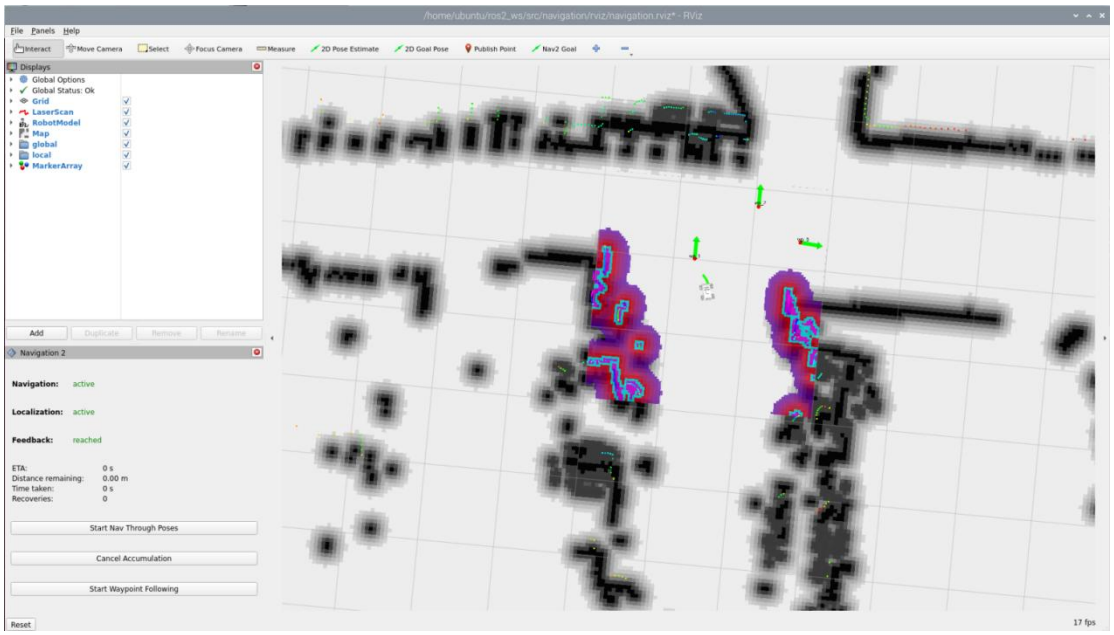
2.2 Multi-Point Navigation

1） Click the "waypoint" button to start multi-point navigation:



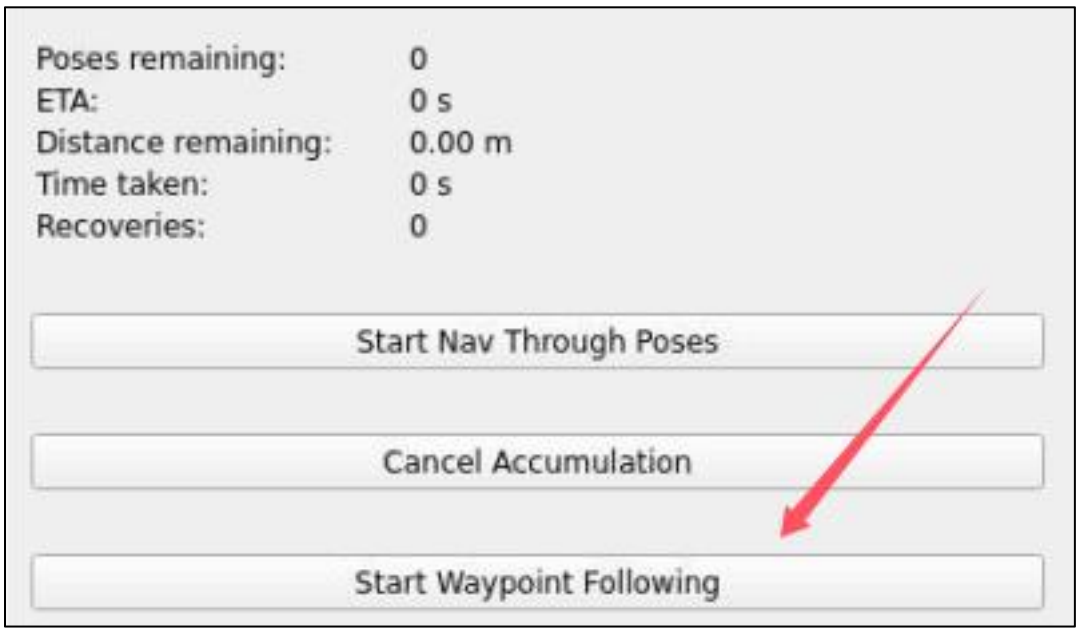
2） Set each navigation point using 

as shown in the figure below:

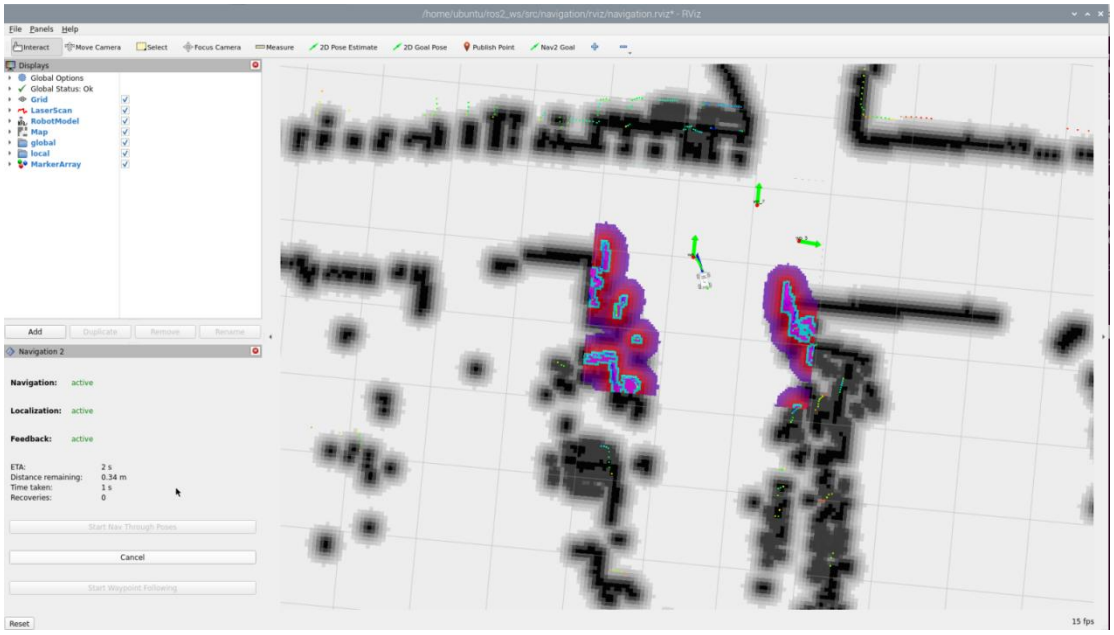


3） Finally, click "Start Nav Through Poses" or "Start Waypoint Following" to start navigation. "Start Nav Through Poses" will control the robot's pose at each navigation point; "Start Waypoint Following" will not focus on the robot's

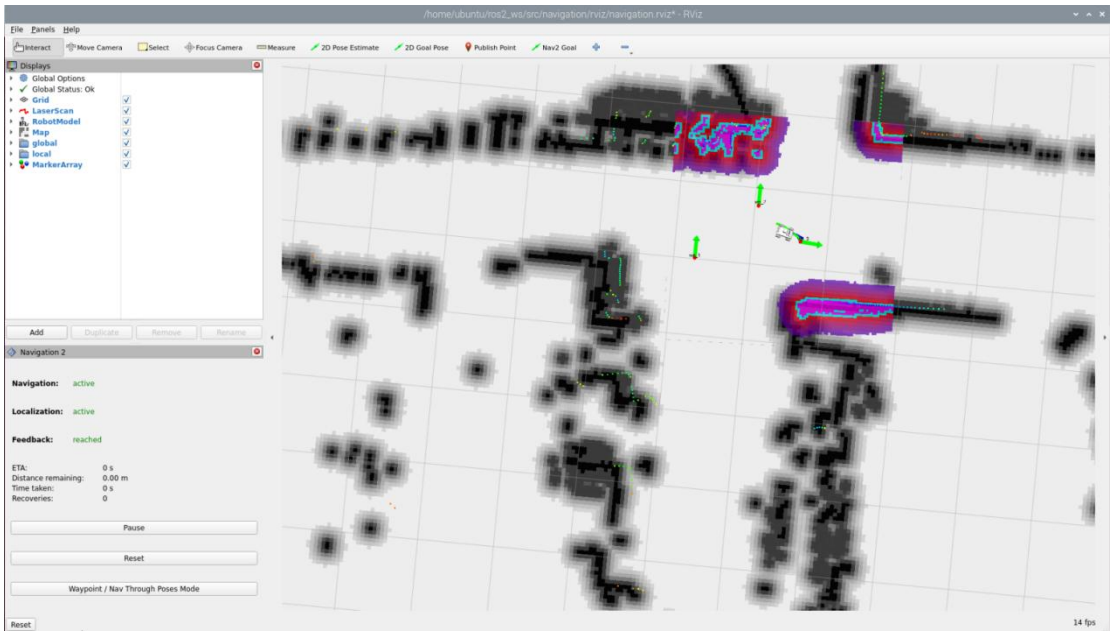
pose at each navigation point.



4） The effect during multi-point navigation is shown in the figure below, with the robot reaching the target points in sequence:



5） The effect after completing multi-point navigation is shown in the figure below:



6） If you want to exit the game, press “Ctrl+C” in the terminal interface.

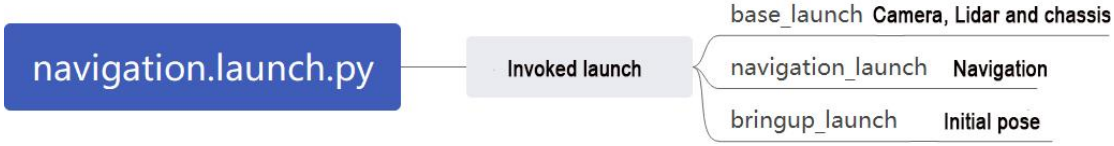
After experiencing the game, you can enable the app service through commands or by restarting the robot. If the app is not enabled, the related app functions will not work. If the robot is restarted, the app will be automatically enabled.

Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

Note: please enter the command in the system path, not in the Docker container.

```
sudo systemctl restart start_node.service
```

3. Launch Description



The path to the launch file is located at:

/home/ubuntu/ros2_ws/src/navigation/launch/navigation.launch.py

```python
import os
from ament_index_python.packages import get_package_share_directory

from launch_ros.actions import PushRosNamespace
from launch import LaunchDescription, LaunchService
from launch.substitutions import LaunchConfiguration
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription, GroupAction, OpaqueFunction, TimerAction

def launch_setup(context):
    compiled = os.environ['need_compile']
    if compiled == 'True':
        slam_package_path = get_package_share_directory('slam')
        navigation_package_path = get_package_share_directory('navigation')
    else:
        slam_package_path = '/home/ubuntu/ros2_ws/src/slam'
        navigation_package_path = '/home/ubuntu/ros2_ws/src/navigation'

    sim = LaunchConfiguration('sim', default='false').perform(context)
    map_name = LaunchConfiguration('map', default='map_01').perform(context)
    robot_name = LaunchConfiguration('robot_name', default=os.environ['HOST']).perform(context)
    master_name = LaunchConfiguration('master_name', default=os.environ['MASTER']).perform(context)
    use_teb = LaunchConfiguration('use_teb', default='false').perform(context)

    sim_arg = DeclareLaunchArgument('sim', default_value=sim)
    map_name_arg = DeclareLaunchArgument('map', default_value=map_name)
    master_name_arg = DeclareLaunchArgument('master_name', default_value=master_name)
    robot_name_arg = DeclareLaunchArgument('robot_name', default_value=robot_name)
    use_teb_arg = DeclareLaunchArgument('use_teb', default_value=use_teb)

    use_sim_time = 'true' if sim == 'true' else 'false'
    use_namespace = 'true' if robot_name != '/' else 'false'
```

◆ Set Paths

Obtain the paths for the “slam” and “navigation” packages.

```python
    if compiled == 'True':
        slam_package_path = get_package_share_directory('slam')
        navigation_package_path = get_package_share_directory('navigation')
    else:
        slam_package_path = '/home/ubuntu/ros2_ws/src/slam'
        navigation_package_path = '/home/ubuntu/ros2_ws/src/navigation'
```

◆ Enable Other Launch Files

“base_launch” for various hardware

“navigation_launch” to start the navigation algorithm

“bringup_launch” to initialize actions

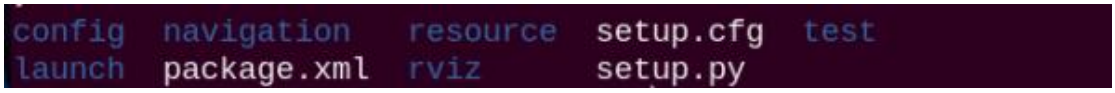
```python
    base_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(slam_package_path, 'launch/include/robot.launch.py')),
        launch_arguments={
            'sim': sim,
            'master_name': master_name,
            'robot_name': robot_name
        }.items(),
    )

    navigation_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(navigation_package_path, 'launch/include/bringup.launch.py')),
        launch_arguments={
            'use_sim_time': use_sim_time,
            'map': os.path.join(slam_package_path, 'maps', map_name + '.yaml'),
            'params_file': os.path.join(navigation_package_path, 'config', 'nav2_params.yaml'),
            'namespace': robot_name,
            'use_namespace': use_namespace,
            'autostart': 'true',
            'use_teb': use_teb,
        }.items(),
    )

    bringup_launch = GroupAction(
     actions=[
         PushRosNamespace(robot_name),
         base_launch,
         TimerAction(
             period=10.0,  # 延时等待其它节点启动好(delay for enabling other nodes)
             actions=[navigation_launch],
         ),
      ]
    )
```

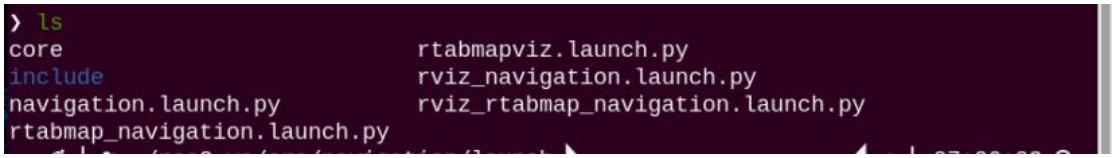
4. Feature Pack Description

The path to the navigation package is: **/home/ubuntu/ros2_ws/src/navigation/**

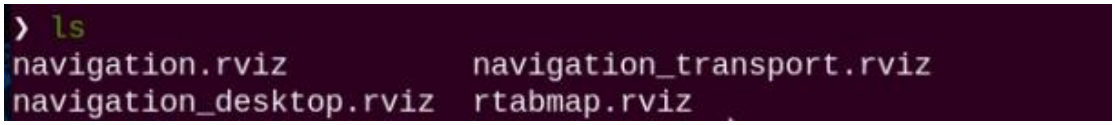


config: Contains configuration parameters related to navigation, as shown below:

launch: Contains launch files related to navigation, including localization, map loading, navigation modes, and simulation model launch files, as shown below:



rviz: Contains parameter loading for the RViz visualization tool, including RViz configuration files for robots using different navigation algorithms and navigation configuration files, as shown below:



Package.xml: Configuration information file for the current package.